AHB Slave Protocol Implementation

Objective

To design and verify an AHB (Advanced High-performance Bus) Slave module using Verilog HDL that performs register-mapped read and write operations according to the AMBA AHB-Lite protocol.

Introduction

The **Advanced High-performance Bus (AHB)** is a high-speed bus protocol defined in ARM's Advanced Microcontroller Bus Architecture (AMBA). It is designed for communication between processors, memories, and high-speed peripherals in System-on-Chip (SoC) designs. Compared to APB, AHB supports higher performance through pipelined transfers, burst transactions, and multiple bus masters. In this implementation, an AHB slave with a 32-word memory is designed to perform basic single read and write operations while responding to valid and invalid address accesses.

Design Implementation

The AHB Slave was implemented using the following steps:

Step 1

Declare all AHB interface signals including clock, reset, address, control, data, and response signals.

Step 2

Declare a 32 × 32-bit memory array to store data.

Step 3

Implement active-low reset logic to initialize memory locations and output signals.

Step 4

Detect a valid AHB transfer using the conditions:

- HSEL = 1
- HREADY = 1
- HTRANS[1] = 1 (NONSEQ or SEQ transfer)

Step 5

Perform a write operation when HWRITE = 1 by storing HWDATA into the selected memory location.

Step 6

Perform a read operation when HWRITE = 0 by placing the stored memory value onto HRDATA.

Step 7

Check the address range. If the address is outside the memory boundary, assert HRESP while returning zero on HRDATA.

Step 8

Generate HREADYOUT to indicate successful completion of the transfer.

Code:

Design.v

```
module ahb_slave(
     input hclk,
     input hresetn,
     input hsel,
     input [31:0] haddr,
     input [1:0] htrans,
     input hwrite,
     input [2:0] hsize,
     input [31:0] hwdata,
     input hready,
     output reg [31:0] hrdata,
     output reg hreadyout,
     output reg hresp
);
reg [31:0] memory [0:31];
integer i;
always @(posedge hclk or negedge hresetn)
```

```
                if(hwrite)
                begin
                    memory[haddr] <= hwdata;
                end
                else
                begin
                    hrdata <= memory[haddr];
                end
            end
            else
            begin
                hrdata <= 32'd0;
                hresp <= 1'b1;
            end
        end
    end
end
```

Tb.v

```
`timescale 1ns/1ps
module ahb_slave_tb;
    reg hclk;
    reg hresetn;
    reg hsel;
    reg [31:0] haddr;
    reg [1:0] htrans;
    reg hwrite;
    reg [2:0] hsize;
    reg [31:0] hwdata;
    reg hready;
    wire [31:0] hrdata;
    wire hreadyout;
    wire hresp;
ahb_slave DUT(
  .hclk(hclk),
  .hresetn(hresetn),
```

```
  .hsel(hsel),
  .haddr(haddr),
  .htrans(htrans),
  .hwrite(hwrite),
  .hsize(hsize),
  .hwdata(hwdata),
  .hready(hready),
  .hrdata(hrdata),
  .hreadyout(hreadyout),
  .hresp(hresp)
);
initial
  hclk = 0;
always #5 hclk = ~hclk;
task ahb_write;
input [31:0] address;
input [31:0] data;
```

```
begin
    @(posedge hclk);
    hsel = 1;
    hready = 1;
    htrans = 2'b10;
    hwrite = 1;
    haddr = address;
    hwdata = data;
    @(posedge hclk);
    hsel = 0;
    hwrite = 0;
    end
    endtask
    task ahb_read;
    input [31:0] address;
    begin
    @(posedge hclk);
```

```
hsel = 1;
hready = 1;
htrans = 2'b10;
hwrite = 0;
haddr = address;
@(posedge hclk);
$display("Time=%0t Address=%0d Data=%h Ready=%b Resp=%b",
$time,haddr,hrdata,hreadyout,hresp);
hsel = 0;
end
endtask
initial
begin
  $dumpfile("dump.vcd");
  $dumpvars(0,ahb_slave_tb);
hresetn = 0;
hsel = 0;
```

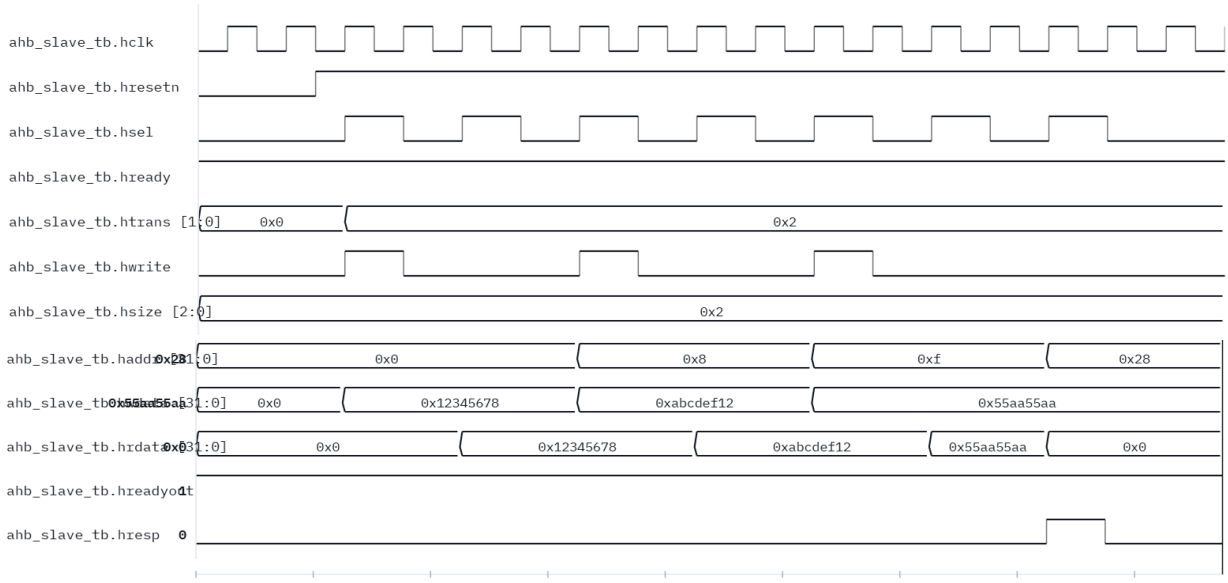
```
haddr = 0;
htrans = 2'b00;
hwrite = 0;
hsize = 3'b010;
hwdata = 0;
hready = 1;
#20;
hresetn = 1;
ahb_write(0,32'h12345678);
ahb_read(0);
```

```
    ahb_write(8,32'hABCDEF12);
    ahb_read(8);
    ahb_write(15,32'h55AA55AA);
    ahb_read(15);
    ahb_read(40);
    #20;
    $finish;
    end
endmodule
```

Output:

```
VCD info: dumpfile dump.vcd opened for output.
Time=55000 Address=0 Data=12345678 Ready=1 Resp=0
Time=95000 Address=8 Data=abcdef12 Ready=1 Resp=0
Time=135000 Address=15 Data=55aa55aa Ready=1 Resp=0
Time=155000 Address=40 Data=00000000 Ready=1 Resp=1
/tb.v:85: $finish called at 175000 (1ps)
timings ms: preprocess=16.0 compile=105 simulate=65.1 vcd-parse=
```

Waveform:



Conclusion

The AHB Slave module was successfully designed and implemented using Verilog HDL. The design supports register-mapped memory with 32 storage locations and performs single read and write transactions according to the AMBA AHB-Lite protocol. The implemented logic correctly detects valid bus transfers, stores and retrieves data from memory, and generates appropriate response signals. The developed testbench verified reset functionality, read/write operations, and invalid address handling. Simulation results matched the expected behavior, confirming the correctness of the AHB slave implementation and demonstrating its suitability for communication with high-speed peripherals in System-on-Chip (SoC) applications.